COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 009B	**SECTION:**	CPE21S4
DATE PERFORMED:	8/8/2026	**DATE SUBMITTED:**	8/11/2026
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __4.2_ Advance QT Widgets

1. PROCEDURE OUTPUT

Act4 > main.py

```python
import sys
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
    QHBoxLayout,QLineEdit, QPushButton, QTableWidget, QTableWidgetItem,
    QComboBox, QDateEdit, QProgressBar, QMenuBar, QMessageBox
)
from PyQt6.QtCore import Qt, QDate

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Task Manager Lite")
        self.setFixedSize(600, 400)

        self.tasks = []

        self.create_menu()
        self.setup_ui()

    def create_menu(self):
        menu_bar = self.menuBar()
        file_menu = menu_bar.addMenu("File")
        exit_action = file_menu.addAction("Exit")
        exit_action.triggered.connect(self.close)

        help_menu = menu_bar.addMenu("Help")
        about_action = help_menu.addAction("About")
        about_action.triggered.connect(lambda: QMessageBox.information(
            self, "About", "Task Manager Lite\nDeveloped by Denzen Igloso"
        ))

```

```
32      def setup_ui(self):
33          main_layout = QHBoxLayout()
34
35          input_layout = QHBoxLayout()
36          self.task_input = QLineEdit()
37          self.task_input.setPlaceholderText("Enter a new task")
38          add_button = QPushButton("Add Task")
39          add_button.clicked.connect(self.add_task)
40          input_layout.addWidget(self.task_input)
41          input_layout.addWidget(add_button)
42
43          self.task_table = QTableWidget()
44          self.task_table.setColumnCount(2)
45          self.task_table.setHorizontalHeaderLabels(["Task", "Status"])
46          self.task_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
```

```
48          controls_layout = QHBoxLayout()
49          self.status_combo = QComboBox()
50          self.status_combo.addItems(["Pending", "In Progress", "Completed"])
51          self.status_combo.currentIndexChanged.connect(self.update_status)
52
53          self.date_edit = QDateEdit()
54          self.date_edit.setDate(QDate.currentDate())
55          self.date_edit.dateChanged.connect(self.set_due_date)
56
57          controls_layout.addWidget(self.status_combo)
58          controls_layout.addWidget(self.date_edit)
59          self.progress_bar = QProgressBar()
60
61          self.progress_bar = QProgressBar()
```

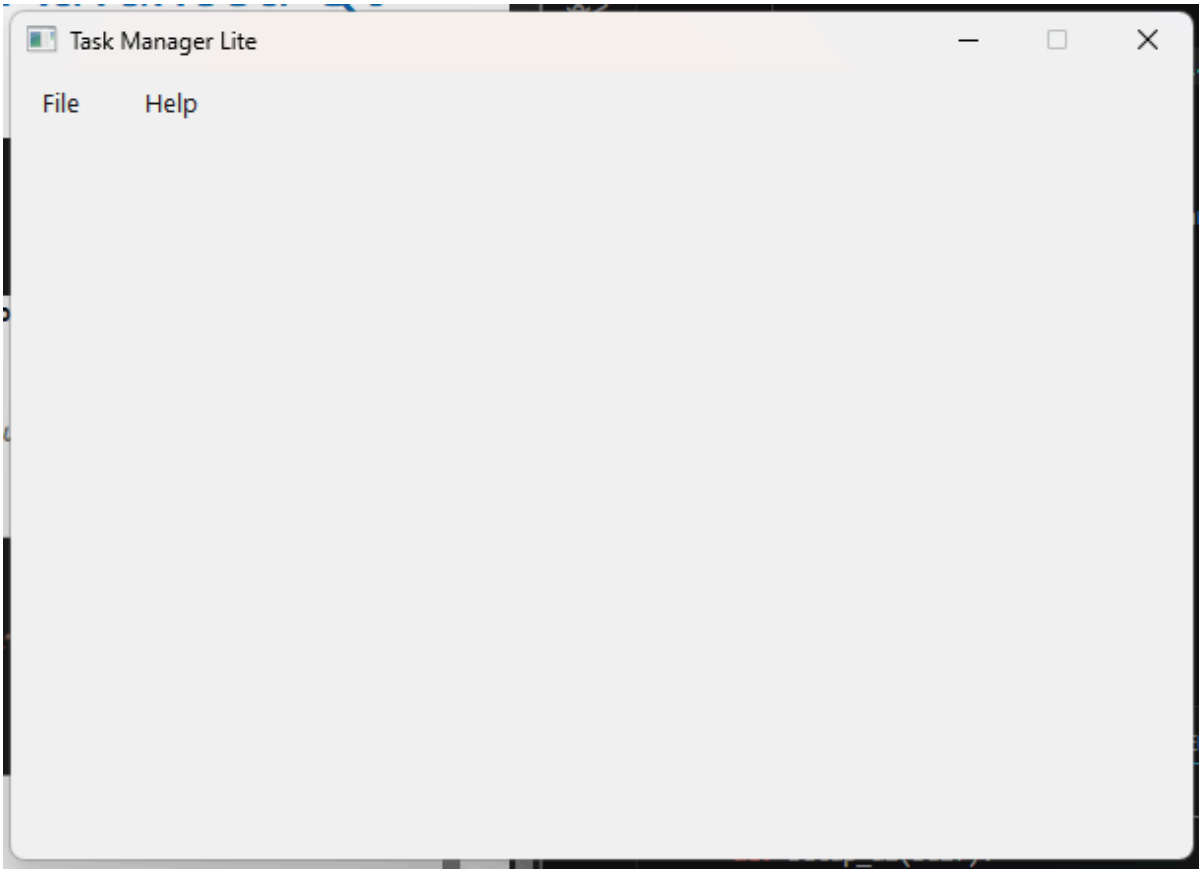
```
        main_layout.addLayout(input_layout)
        main_layout.addWidget(self.task_table)
        main_layout.addLayout(controls_layout)
        main_layout.addWidget(self.progress_bar)

        container = QWidget()
        container.setLayout(main_layout)
        self.setCentralWidget(container)

    def add_task(self):
        task_text = self.task_input.text().strip()
        if task_text:
            row = self.task_table.rowCount()
            self.task_table.insertRow(row)
            self.task_table.setItem(row, 0, QTableWidgetItem(task_text))
            self.task_table.setItem(row, 1, QTableWidgetItem("Pending"))
            self.task_input.clear()
            self.update_progress()
```

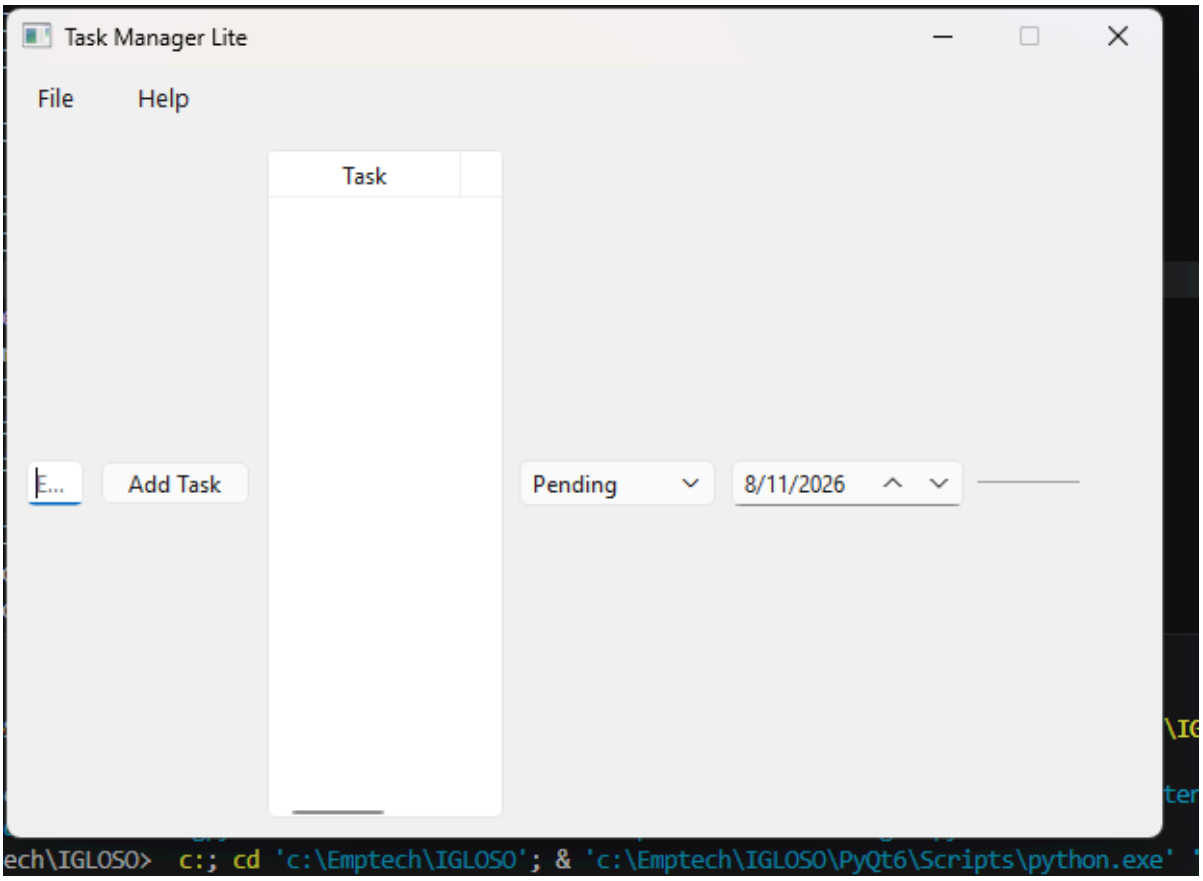
```

    def update_status(self):
        row = self.task_table.currentRow()
        if row >= 0:
            new_status = self.status_combo.currentText()
            self.task_table.setItem(row, 1, QTableWidgetItem(new_status))
            self.update_progress()

    def set_due_date(self):
        row = self.task_table.currentRow()
        if row >= 0:
            due_date = self.date_edit.date().toString(Qt.DateFormat.ISODate)
            QMessageBox.information(self, "Due Date Set", f"Task due date: {due_date}")

```

```
    def update_progress(self):
        total_tasks = self.task_table.rowCount()
        if total_tasks == 0:
            self.progress_bar.setValue(0)
            return
        completed_tasks = sum(
            1 for i in range(total_tasks)
            if self.task_table.item(i, 1).text() == "Completed"
            )
        progress = int((completed_tasks / total_tasks) * 100)
        self.progress_bar.setValue(progress)

```

```python
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```

OUTPUT Procedure



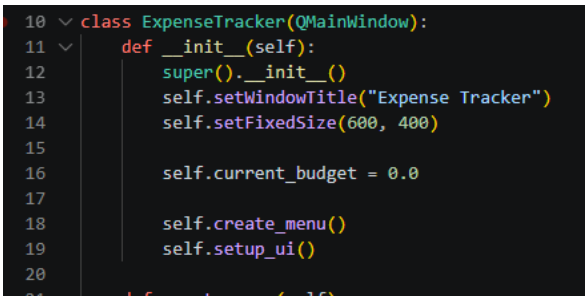


Observation

- For this procedure, I successfully follow and executed the given code and generated the output shown above, which will serve as the basis for the supplementary activity. after showing the output, I noticed several alignment issues within the applications user interface layout.

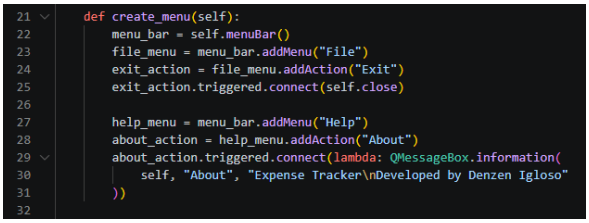
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Expense Tracker Code	Analysis
main.py Expense_tracker.py Act4 > Expense_tracker.py > ExpenseTracker > setup_ui 1 import sys 2 from PyQt6.QtWidgets import (3 QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, 4 QLineEdit, QPushButton, QTableWidget, QTableWidgetItem, 5 QComboBox, QProgressBar, QMessageBox, QHeaderView 6) 7 from PyQt6.QtCore import Qt 8	This is the declaration of the



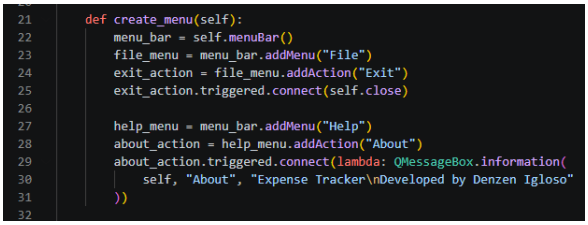
```
class ExpenseTracker(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Expense Tracker")
        self.setFixedSize(600, 400)

        self.current_budget = 0.0

        self.create_menu()
        self.setup_ui()
```

This is where i import all the needed modules and widgets from pyqt6 so that i can create the window, buttons, input fields, and the table for the application.



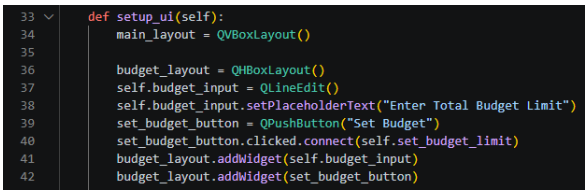
```
    def create_menu(self):
        menu_bar = self.menuBar()
        file_menu = menu_bar.addMenu("File")
        exit_action = file_menu.addAction("Exit")
        exit_action.triggered.connect(self.close)

        help_menu = menu_bar.addMenu("Help")
        about_action = help_menu.addAction("About")
        about_action.triggered.connect(lambda: QMessageBox.information(
            self, "About", "Expense Tracker\nDeveloped by Denzen Igloso"
        ))
```

This is where i set up the title of the window and the size of it. Also i declared the current_budget to 0.0 for the preparation of budget limit, and then called the methods for menu and ui.

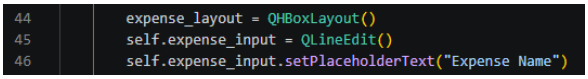


```
    def create_menu(self):
        menu_bar = self.menuBar()
        file_menu = menu_bar.addMenu("File")
        exit_action = file_menu.addAction("Exit")
        exit_action.triggered.connect(self.close)

        help_menu = menu_bar.addMenu("Help")
        about_action = help_menu.addAction("About")
        about_action.triggered.connect(lambda: QMessageBox.information(
            self, "About", "Expense Tracker\nDeveloped by Denzen Igloso"
        ))
```

This is where i create the menu bar at the top, it has a file menu to close the program and a help menu that shows an about popup dialog with my name on it. Exactly like the one on the procedure.

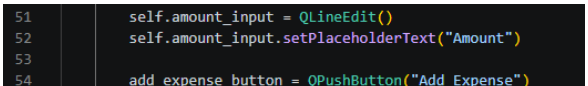


```
    def setup_ui(self):
        main_layout = QVBoxLayout()

        budget_layout = QHBoxLayout()
        self.budget_input = QLineEdit()
        self.budget_input.setPlaceholderText("Enter Total Budget Limit")
        set_budget_button = QPushButton("Set Budget")
        set_budget_button.clicked.connect(self.set_budget_limit)
        budget_layout.addWidget(self.budget_input)
        budget_layout.addWidget(set_budget_button)
```



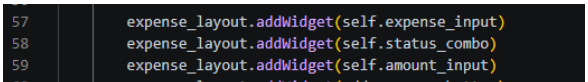
```
        expense_layout = QHBoxLayout()
        self.expense_input = QLineEdit()
        self.expense_input.setPlaceholderText("Expense Name")
```

```
self.status_combo = QComboBox()
self.status_combo.addItems(["Food", "Transportation", "Rent", "Utilities", "Entertainment", "Others"])
```

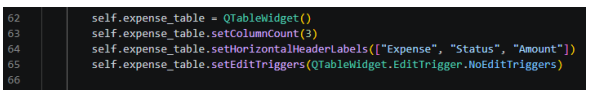


```
        self.amount_input = QLineEdit()
        self.amount_input.setPlaceholderText("Amount")

        add_expense_button = QPushButton("Add Expense")
        add_expense_button.clicked.connect(self.add_expense)
```

This part is where i set up the input layout for setting the budget limit and the expense inputs like the expense name, category dropdown, amount box, and the add button.



```
        expense_layout.addWidget(self.expense_input)
        expense_layout.addWidget(self.status_combo)
        expense_layout.addWidget(self.amount_input)
        expense_layout.addWidget(add_expense_button)
```



```
        self.expense_table = QTableWidget()
        self.expense_table.setColumnCount(3)
        self.expense_table.setHorizontalHeaderLabels(["Expense", "Status", "Amount"])
        self.expense_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
```

This is where i created the table widget with 3 columns, the progress bar, and the reset button. Then i add all of them into the main layout to display vertically.

<table>
<tr><td>

```
67          self.progress_bar = QProgressBar()
68
69          reset_button = QPushButton("Reset All Expenses")
70          reset_button.clicked.connect(self.reset_expenses)
71
72          main_layout.addLayout(budget_layout)
73          main_layout.addLayout(expense_layout)
74          main_layout.addWidget(self.expense_table)
75          main_layout.addWidget(self.progress_bar)
76          main_layout.addWidget(reset_button)
77
78          container = QWidget()
79          container.setLayout(main_layout)
80          self.setCentralWidget(container)
81
```

</td><td></td></tr>
<tr><td>

```
83      def set_budget_limit(self):
84          raw_text = self.budget_input.text().strip()
85          if raw_text != "":
86              self.current_budget = float(raw_text)
87              QMessageBox.information(self, "Budget Set", f"Budget limit updated to {self.current_budget:.2f}")
88              self.update_progress()
```

</td><td>This method gets the text inside the budget input box, convert it to float, and set it as my current budget. It also show a message box that the budget is set and update the progress bar.</td></tr>
<tr><td>

```
90      def add_expense(self):
91          expenses = self.expense_input.text().strip()
92          status = self.status_combo.currentText()
93          amount_text = self.amount_input.text().strip()
94
95          if expenses != "" and amount_text != "":
96              amount = float(amount_text)
97              row = self.expense_table.rowCount()
98              self.expense_table.insertRow(row)
99
100             self.expense_table.setItem(row, 0, QTableWidgetItem(expenses))
101             self.expense_table.setItem(row, 1, QTableWidgetItem(status))
102             self.expense_table.setItem(row, 2, QTableWidgetItem(f"{amount:.2f}"))
103
104             self.expense_input.clear()
105             self.amount_input.clear()
106
107             self.update_progress()
108
```

</td><td>This function is for adding the expense to the table. It checks if the fields are not empty, then insert a new row in the table with the values i typed and clear the inputs after.</td></tr>
<tr><td>

```
109     def update_progress(self):
110         if self.current_budget <= 0:
111             self.progress_bar.setValue(0)
112             return
113
114         total_spent = 0.0
115         for i in range(self.expense_table.rowCount()):
116             item = self.expense_table.item(i, 2)
117             if item is not None:
118                 total_spent += float(item.text())
119
120         percentage = int((total_spent / self.current_budget) * 100)
121
122         self.progress_bar.setValue(percentage)
123
```

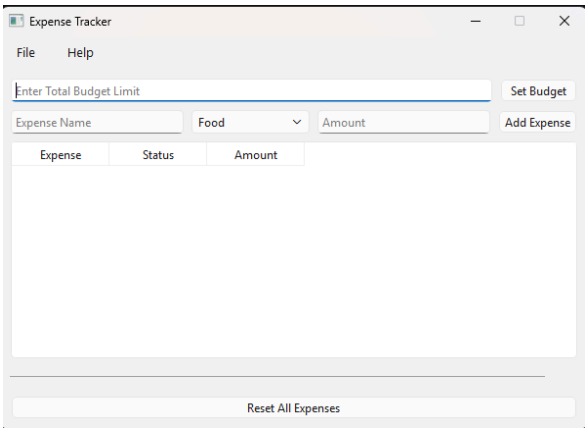
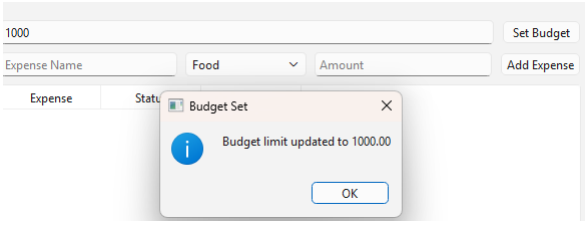
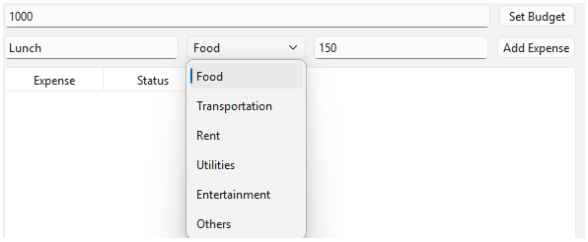
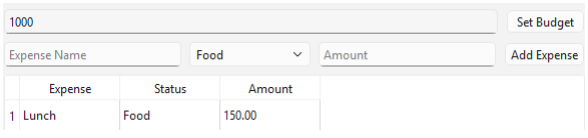
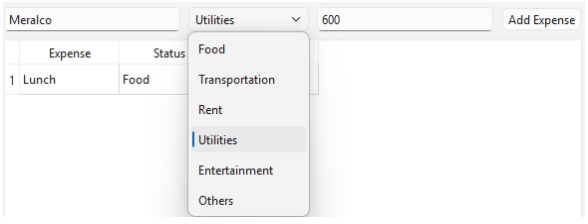
</td><td>This function calculates the total amount spent by looping through column 2 of the table. Then it calculates the percentage based on the budget limit and update the progress bar.</td></tr>
<tr><td>

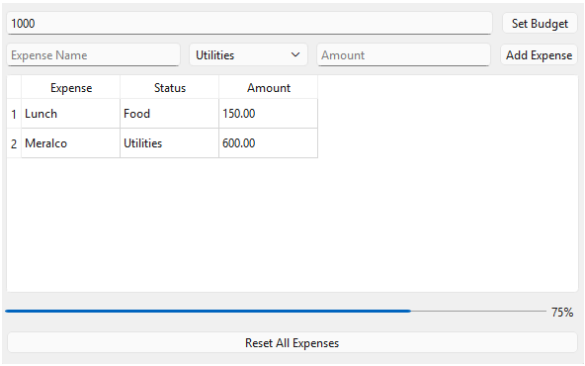
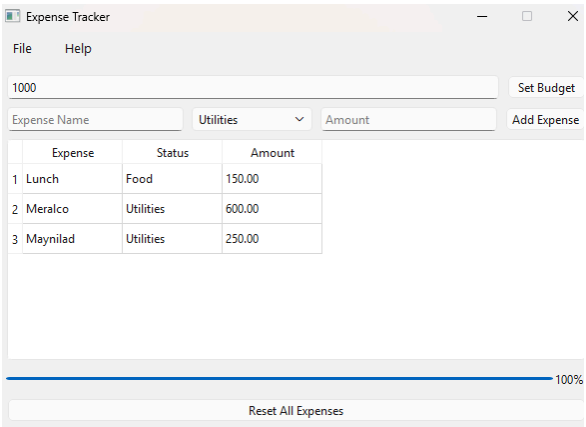
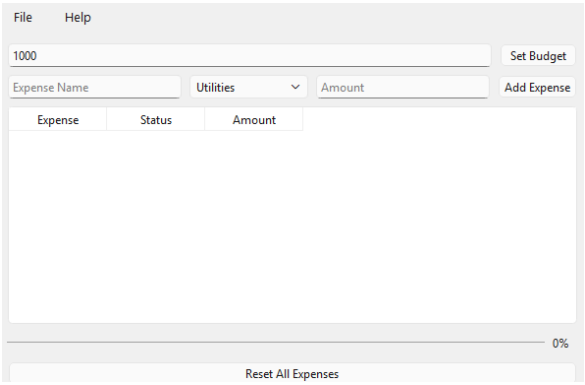
```
124     def reset_expenses(self):
125         self.expense_table.setRowCount(0)
126         self.update_progress()
```

</td><td>This is where i clear all the rows in the table by setting the row count back to 0, so that it will also reset the progress bar back to 0percent.</td></tr>
<tr><td>

```
129 if __name__ == "__main__":
130     app = QApplication(sys.argv)
131     window = ExpenseTracker()
132     window.show()
133     sys.exit(app.exec())
```

</td><td>This is the main execution code that runs the app, create the expense tracker window object, and show it on the screen.</td></tr>
</table>

Output	Observation
	This is the output after a run the code
	This is where i set the budget, for example 1000. Then click the set budget and the message will pop up on the screen that says the limit updated.
	This is where you put the type of expense you have, for example Lunch, you can select on the categories, like Food. then set the amount of that food and then click the add Expense.
	This is the result of that after you clicked the add expense. Then it will automatically clear the expense name and the amount using the clear() function.
	Another add expense, meralco and the category is utilities and the amount is 600.

	It shows the 2 expenses added and at the bottom where the progress bar is located, it is indicated where when the expense increases it will also increase the progress bar to the100 percent which is the budget limit set by the user.
	As you can see, the progress bar reach the 100 percent after the expenses reach the budget limit of 1000.
	After i click the reset all expenses at the bottom of the progress bar, all of the expenses are deleted and also the progress bar resets to 0 percent.

Conclusion

- In conclusion, i was able to create this expense tracker application with python and pyqt6 widget in order to track expenses. I have gained knowledge in connecting button signals to slot methods and computing the total expenses in order to make a dynamic updating of progress bar. Through this program, I am now familiar with vertical and horizontal layout in particular on how to work with the table data.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to assist me with some of the codes especially on the vertical and horizontal and i ask about how to remove the inputs after the expense was added and it is actually the function clear() and also i ask ai to help me understand how to analyze the errors on my code when im stuck.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.